

FAZA 2 - Laboratorul 4

Diagrame de secvență și rafinarea diagramei de clase

Aplicație: AutoService Manager - Iterația 1

Student: Lesan Dominique-Arcadii | UBB Informatică, Anul II | Limbaj: Java

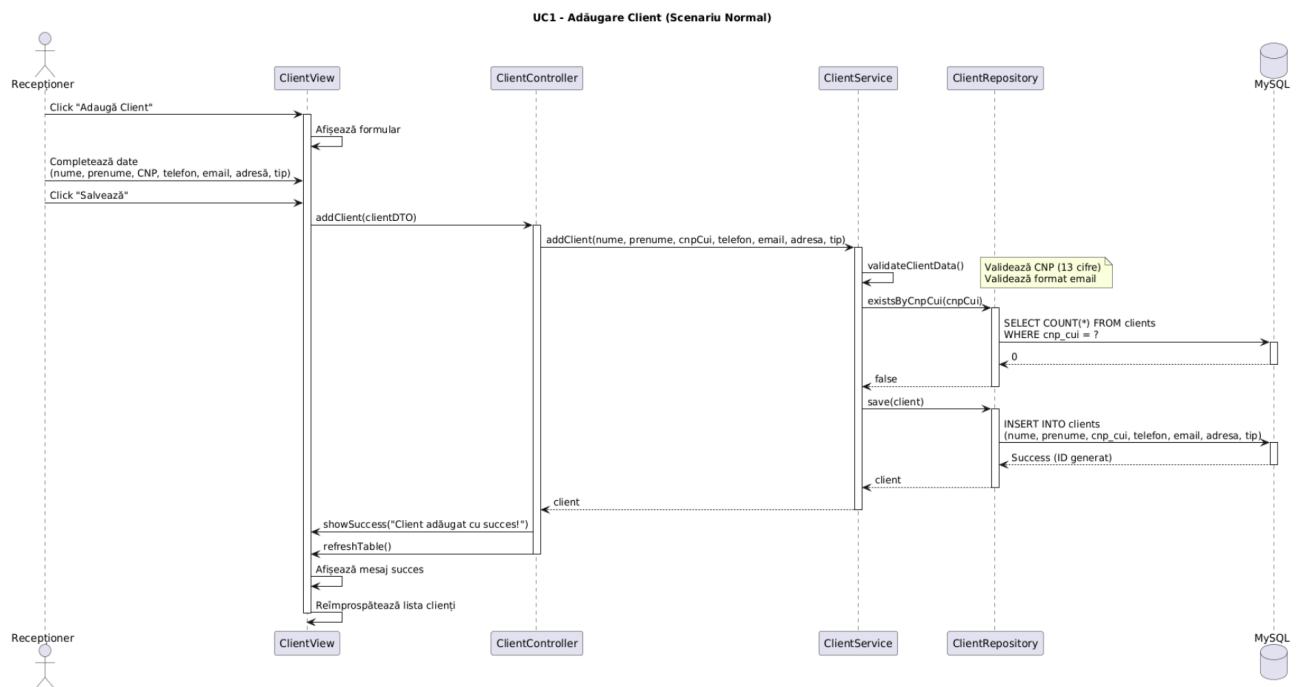
Introducere

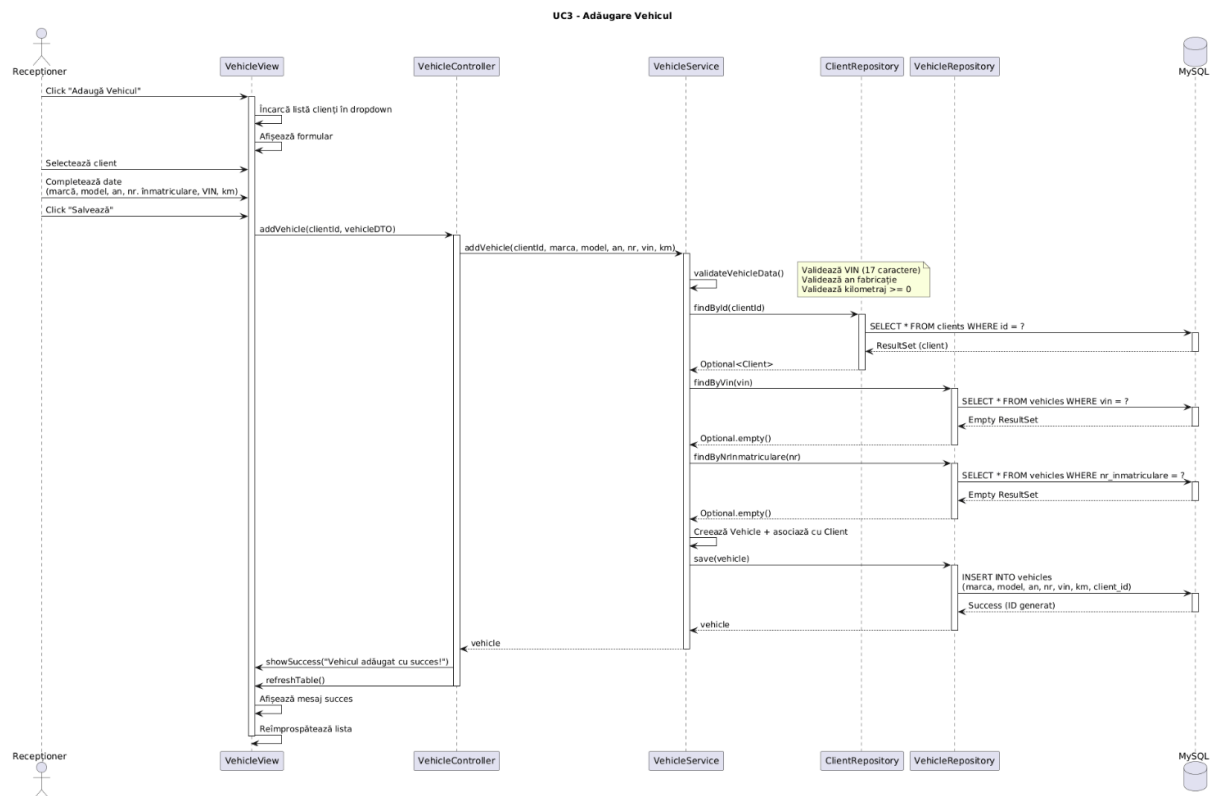
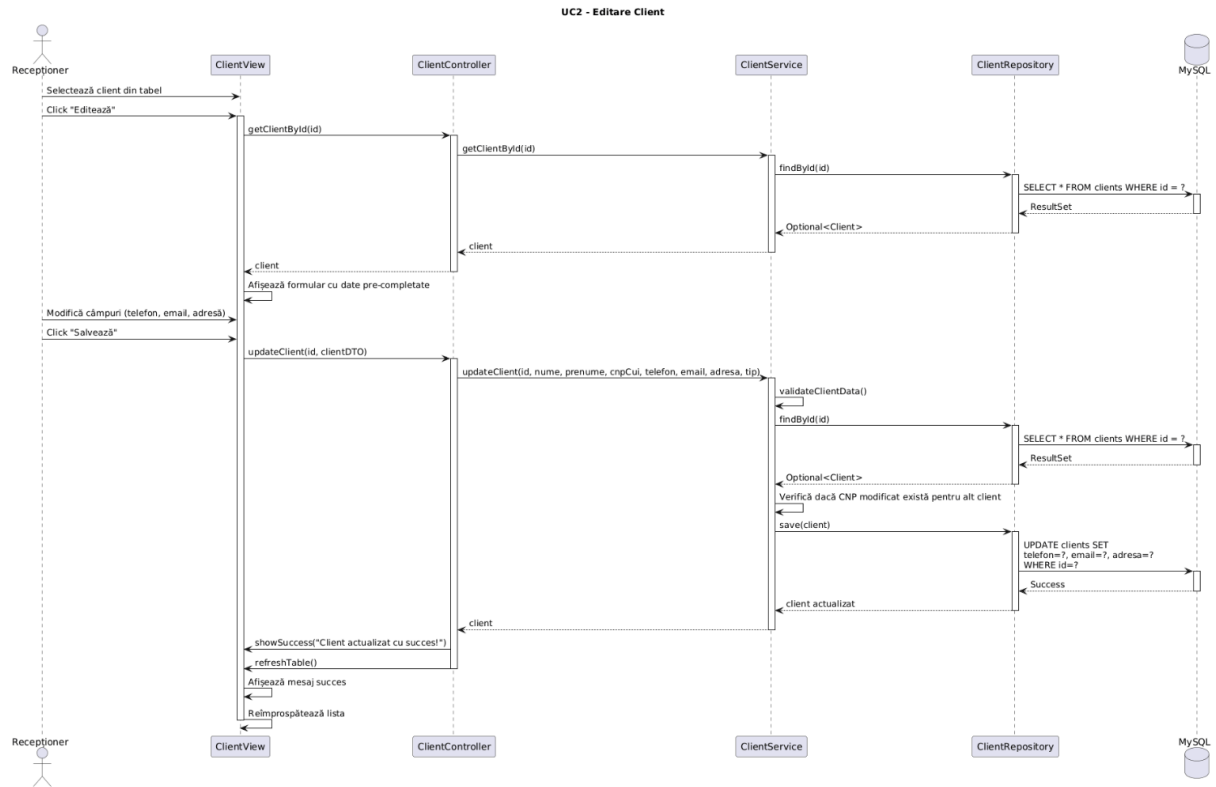
În Laboratorul 4 se realizează modelarea dinamică a aplicației prin diagrame de secvență pentru cazurile de utilizare din Iterația 1, precum și rafinarea diagramei de clase conceptuale cu includerea entităților din domeniul soluției (controllere, servicii, repository-uri).

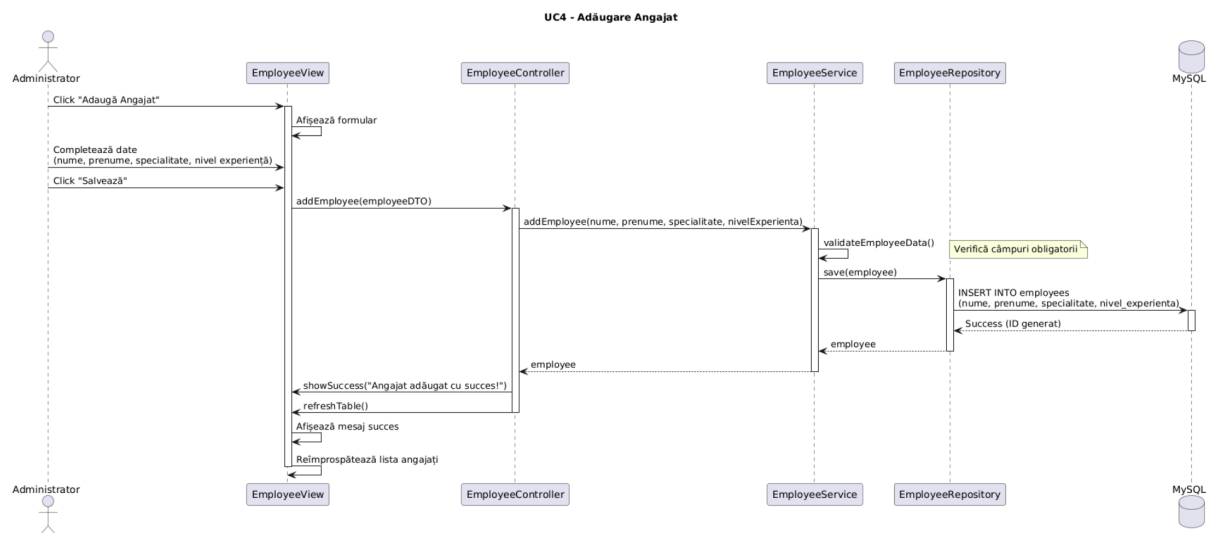
Iterația 1 cuprinde:

- F1 - Gestionare clienți (adăugare, editare, ștergere, căutare)
- F2 - Gestionare vehicule (evidența mașinilor asociate clienților)
- F7 (parțial) - Gestionare angajați (doar evidența mecanicilor de bază)

1. Diagrame de secvență pentru Iterația 1







1.1 UC1 - Înregistrare client (scenariu normal)

Actori: Receptor

Flux principal:

1. Receptorul deschide fereastra de gestionare clienți și apasă butonul "Adăugă client"
2. Sistemul afișează formularul cu câmpurile: nume, prenume/denumire, CNP/CUI, telefon, email, adresă, tip (PF/PJ)
3. Receptorul completează toate câmpurile obligatorii și apasă "Salvează"
4. ClientController primește cererea și apelează ClientService.addClient(clientDTO)
5. ClientService validează datele (verifică câmpuri obligatorii, format CNP/CUI valid)
6. ClientService apelează ClientRepository.existsByCnpCui(cnpCui) pentru a verifica duplicatele
7. Dacă nu există duplicat, ClientRepository.save(client) salvează clientul în baza de date
8. ClientService returnează success și clientul salvat către ClientController
9. ClientController notifică interfața să afișeze mesajul "Client adăugat cu succes"
10. Lista de clienți se reîmprospătează automat cu noul client adăugat

Scenarii alternative:

- E1: Câmpuri obligatorii goale - ClientService aruncă ValidationException, interfața afișează mesaj de eroare specific pentru fiecare câmp
- E2: CNP/CUI duplicat - ClientRepository.existsByCnpCui() returnează true, ClientService aruncă DuplicateClientException, interfața afișează "Client cu acest CNP/CUI există deja în sistem"

Entități implicate:

Componentă	Responsabilitate
ClientView (UI)	Afișare formular, colectare date de la utilizator, afișare mesaje
ClientController	Primește cererea din UI, coordonează fluxul, gestionează răspunsurile
ClientService	Logică de business: validare date, verificare duplicate, orchestrare operații
ClientRepository	Acces la baza de date: salvare, verificare existență, interogări
Client (entitate)	Model de date JPA: id, nume, prenume, cnpCui, telefon, email, adresă, tip

1.2 UC - Editare client

Flux principal:

- 1. Receptorul selectează un client din listă și apasă butonul "Editează"
- 2. ClientController apelează ClientService.getClientById(id) pentru a încărca datele
- 3. ClientService apelează ClientRepository.findById(id) care returnează Optional<Client>
- 4. Sistemul afișează formularul cu datele clientului pre-completate
- 5. Receptorul modifică câmpurile dorite și apasă "Salvează"
- 6. ClientController trimite datele actualizate către ClientService.updateClient(id, clientDTO)
- 7. ClientService validează datele și verifică dacă CNP/CUI modificat nu există deja pentru alt client
- 8. ClientRepository.save(client) actualizează înregistrarea în baza de date
- 9. Interfața afișează "Client actualizat cu succes" și reîmprospătează lista

1.3 UC - Adăugare vehicul

Flux principal:

- 1. Receptorul selectează un client și apasă "Adaugă vehicul"
- 2. VehicleController afișează formularul cu câmpurile: marcă, model, an fabricație, număr înmatriculare, VIN, kilometraj
- 3. Receptorul completează datele și apasă "Salvează"
- 4. VehicleController primește cererea și apelează VehicleService.addVehicle(clientId, vehicleDTO)
- 5. VehicleService validează datele (verifică format VIN, număr înmatriculare valid)
- 6. VehicleService apelează ClientRepository.findById(clientId) pentru a verifica existența clientului
- 7. VehicleService creează entitatea Vehicle și o asociază cu clientul
- 8. VehicleRepository.save(vehicle) salvează vehiculul în baza de date
- 9. Interfața afișează "Vehicul adăugat cu succes" și actualizează lista de vehicule a clientului

Entități implicate:

Componentă	Responsabilitate
VehicleView (UI)	Afișare formular vehicul, colectare date, afișare mesaje
VehicleController	Coordonează operațiile cu vehicule
VehicleService	Validare date vehicul, verificare VIN/număr înmatriculare, logică business
VehicleRepository	Operații CRUD pentru vehicule
Vehicle (entitate)	Model JPA: id, marca, model, anFabricatie, nrInmatriculare, vin, kilometraj, clientId

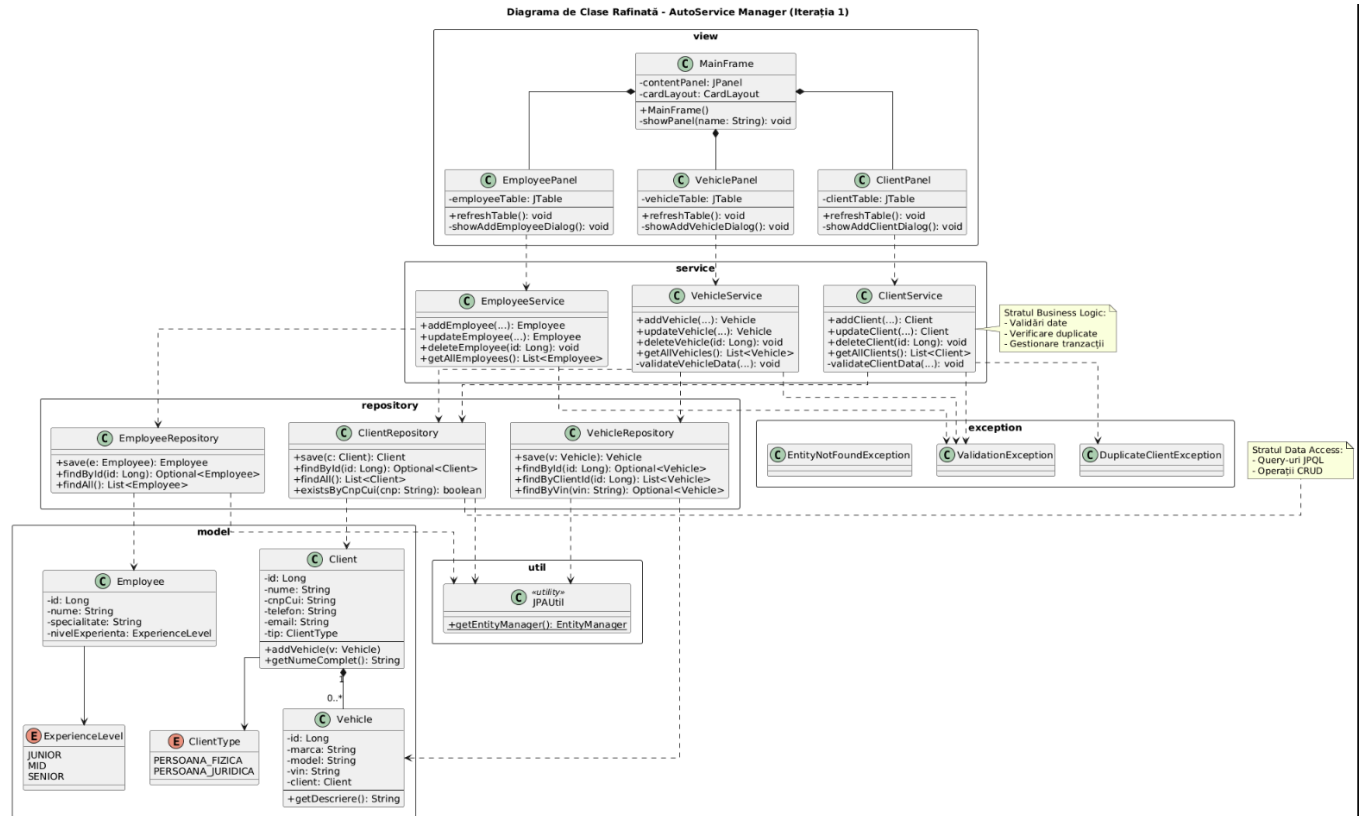
1.4 UC - Adăugare angajat/mecanic (F7 parțial)

Flux principal:

- 1. Administratorul deschide secțiunea Angajați și apasă "Adaugă angajat"
- 2. EmployeeController afișează formularul cu câmpurile: nume, prenume, specialitate, nivel experiență
- 3. Administratorul completează datele și apasă "Salvează"
- 4. EmployeeController apelează EmployeeService.addEmployee(employeeDTO)
- 5. EmployeeService validează datele (verifică câmpuri obligatorii)
- 6. EmployeeRepository.save(employee) salvează angajatul în baza de date
- 7. Interfața afișează "Angajat adăugat cu succes" și actualizează lista de angajați

Notă: În iterația 1 se implementează doar CRUD-ul de bază pentru angajați. Funcționalitățile avansate (asociere la reparații, statistici performanță) se implementează în iterația 3.

2. Diagrama de clase rafinată - Iterația 1



2.1 Arhitectura aplicației

Aplicația AutoService Manager utilizează arhitectura în 3 straturi (three-tier architecture) cu următoarele componente:

Presentation Layer (UI)

- Conține clasele View și Controller care gestionează interfața grafică cu utilizatorul.

Business Logic Layer (Service)

- Conține clasele Service care implementează logica de business: validări, calcule, orchestrare operații.

Data Access Layer (Repository)

- Conține interfețele Repository care gestionează accesul la baza de date folosind JPA/Hibernate.

2.2 Clasele din domeniul soluției - Iterația 1

2.2.1 Pachetul model (entități JPA)

Clasă	Atribute principale	Descriere
Client	id: Long nume: String prenume: String cnpCui: String telefon: String email: String adresa: String tip: ClientType	Entitate JPA pentru clienți. ClientType este enum {PERSOANA_FIZICA, PERSOANA_JURIDICA}

Clasă	Atribute principale	Descriere
Vehicle	id: Longmarca: Stringmodel: StringanFabricatie: IntegernrInmatriculare: Stringvin: Stringkilometraj: Integerclient: Client	Entitate JPA pentru vehicule. Relație @ManyToOne cu Client
Employee	id: Longnume: Stringprenume: Stringspecialitate: StringnivelExperienta: ExperienceLevel	Entitate JPA pentru angajați/mecanici. ExperienceLevel este enum {JUNIOR, MID, SENIOR}

2.2.2 Pachetul repository (acces date)

Interfață	Metode principale
ClientRepository	extends JpaRepository<Client, Long>+ existsByCnpCui(String): boolean+ findByTip(ClientType): List<Client>+ searchByNume(String): List<Client>
VehicleRepository	extends JpaRepository<Vehicle, Long>+ findByClientId(Long): List<Vehicle>+ findByVin(String): Optional<Vehicle>+ findByNrInmatriculare(String): Optional<Vehicle>
EmployeeRepository	extends JpaRepository<Employee, Long>+ findBySpecialitate(String): List<Employee>+ findByNivelExperienta(ExperienceLevel): List<Employee>

2.2.3 Pachetul service (logică business)

Clasă	Metode principale
ClientService	+ addClient(ClientDTO): Client+ updateClient(Long, ClientDTO): Client+ deleteClient(Long): void+ getClientById(Long): Client+ getAllClients(): List<Client>+ searchClients(String): List<Client>- validateClient(ClientDTO): void- checkDuplicate(String): boolean
VehicleService	+ addVehicle(Long, VehicleDTO): Vehicle+ updateVehicle(Long, VehicleDTO): Vehicle+ deleteVehicle(Long): void+ getVehicleById(Long): Vehicle+ getVehiclesByClient(Long): List<Vehicle>- validateVehicle(VehicleDTO): void- validateVIN(String): boolean
EmployeeService	+ addEmployee(EmployeeDTO): Employee+ updateEmployee(Long, EmployeeDTO): Employee+ deleteEmployee(Long): void+ getEmployeeById(Long): Employee+ getAllEmployees(): List<Employee>- validateEmployee(EmployeeDTO): void

2.2.4 Pachetul controller (coordonare UI-Service)

Clasă	Responsabilitate
ClientController	Primește acțiuni din ClientView (add, edit, delete, search), apelează ClientService, gestionează excepțiile și actualizează UI-ul
VehicleController	Coordonează operațiile cu vehicule, apelează VehicleService, gestionează UI pentru vehicule
EmployeeController	Coordonează operațiile cu angajați, apelează EmployeeService

2.2.5 Pachetul view (interfață grafică)

Clasă	Componente UI
ClientView	JPanel cu JTable pentru listă clienți, JButton (add, edit, delete, search), JDialog pentru formular add/edit cu JTextField pentru fiecare câmp
VehicleView	JPanel cu JTable pentru listă vehicule, JButton pentru operații CRUD, JComboBox pentru selectare client, JDialog formular
EmployeeView	JPanel cu JTable pentru listă angajați, JButton pentru operații CRUD, JDialog formular cu JComboBox pentru specialitate și nivel experiență
MainFrame	JFrame principal cu JMenuBar și CardLayout pentru navigare între module (Dashboard, Clienți, Vehicule, Angajați)

2.2.6 Pachetul dto (Data Transfer Objects)

Clasă DTO	Utilizare
ClientDTO	Transfer date între UI și Service pentru clienți. Conține aceleași câmpuri ca entitatea Client dar fără relații JPA
VehicleDTO	Transfer date pentru vehicule, include clientId în loc de referință Client
EmployeeDTO	Transfer date pentru angajați

2.3 Relații între clase - Iterația 1

Dependențe între straturi:

- ClientView → ClientController → ClientService → ClientRepository → Client (entitate)
- VehicleView → VehicleController → VehicleService → VehicleRepository → Vehicle (entitate)
- EmployeeView → EmployeeController → EmployeeService → EmployeeRepository → Employee (entitate)

Relații între entități de date:

- Client 1 ↔ * Vehicle (un client poate avea mai multe vehicule)
- Relația este bidirecțională cu @OneToMany în Client și @ManyToOne în Vehicle
- Employee este independent în iterația 1 (relația cu FisaReparatie se adaugă în iterația 2)

3. Tehnologii și framework-uri utilizate

Tehnologie	Utilizare
Java 17 (LTS)	Limbaj de programare principal
Java Swing	Framework pentru interfața grafică desktop (JFrame, JPanel, JTable, JDialog)
JPA (Java Persistence API)	Specificație pentru mapare obiect-relațional (ORM)
Hibernate	Implementare JPA pentru gestionare bază de date
MySQL / PostgreSQL	Bază de date relațională pentru persistența datelor
Maven / Gradle	Build tool și dependency management
JUnit 5	Framework pentru teste unitare (acoperire minim 60%)
Log4j / SLF4J	Sistem de logging pentru monitorizare și debugging

4. Concluzii Laboratorul 4

În acest laborator au fost realizate:

- Modelarea dinamică completă a scenariilor principale din Iterația 1 prin diagrame de secvență pentru UC: Adăugare client, Editare client, Adăugare vehicul, Adăugare angajat
- Rafinarea diagramei de clase conceptuale cu includerea tuturor claselor din domeniul soluției: View, Controller, Service, Repository, DTO
- Definirea clară a arhitecturii în 3 straturi (Presentation, Business Logic, Data Access) și a dependențelor dintre componente
- Specificarea tehnologiilor și framework-urilor utilizate pentru implementare (Java 17, Swing, JPA/Hibernate, MySQL)